

Smart Contract Synthesis via Multi-Modal Specifications

Anonymous Author(s)

Abstract

Smart contracts manage substantial digital assets on the blockchains. Given their transparency and immutability, ensuring their correctness before deployment is essential. While verification tools offer assistance, the iterative nature of the process presents hurdles, especially for less experienced developers. In this paper, we explore a paradigm shift in smart contract development: writing high-level specifications instead of low-level code. To realize this goal, we propose SmartSpec, a smart contract synthesizer that automatically generates correct implementation from functional and safety specifications. To avoid overly restrictive outcomes, SmartSpec employs maximally permissive synthesis, ensuring contracts preserve all safety requirements while remaining as permissive as possible. Its novel algorithm combines deductive and inductive synthesis techniques, balancing precision and generalization. Evaluation on **XX** representative smart contracts demonstrates the effectiveness and efficiency of SmartSpec, synthesizing correct smart contracts in seconds to minutes. The generated smart contracts exhibit comparable gas efficiency with hand-written reference implementations.

1 Introduction

Smart contracts have gained significant traction in the finance and business sectors, where they manage substantial assets and facilitate high-value transactions. However, these contracts are frequently targeted by attacks, leading to considerable financial losses [12, 21]. As a result, ensuring the correctness of smart contracts is critically important. Although various verification tools have been developed to enhance smart contract security [8, 23, 24, 32, 35], the process often requires multiple iterations between the programmer and the tool to produce high-quality code. Moreover, many smart contract designers lack expertise in formal methods, making it difficult for them to interpret and act on the feedback provided by such tools.

This paper explores a synthesis-oriented approach to smart contract development. Instead of manually implementing and repeatedly verifying code, developers provide formal specifications of the desired contract behaviors, and an automated algorithm generates implementations that satisfy these specifications. This shifts the effort from low-level programming and debugging to high-level specification design, which is more declarative, easier to revise, and more flexible in capturing corner cases directly.

To this end, we study the smart contract synthesis problem. How should smart contracts be specified in a precise yet practical way? Given such specifications, can we automatically generate a provably correct implementation? Addressing these questions requires overcoming several key challenges.

Challenge 1: precise yet practical specification. Synthesizing correct smart contracts requires specifications that are both precise and practical. Input-output examples have proven useful in domains like string manipulation [19, 27], program analysis [28, 29], and query generation [33], but they offer no formal guarantees, which

is unacceptable for security-critical contracts. Formal specifications [10, 37] offer strong correctness guarantees but demand expertise and manual effort. Even lightweight alternatives like DeCon [9] fall short for properties involving long-term behaviors [8, 11, 26].

Multiple modalities can be both precise and practical. We observe that combining modalities, more specifically, relational and temporal logic, can cover both local function behaviors and global safety properties over execution histories. We adopt *first-order relational logic* [9] to define contract state through inference over transaction histories, supporting precise reasoning about function-level correctness. We further integrate *temporal logic* [26] to express safety properties across transaction sequences. This unified framework captures both local and global correctness. As shown in Section 2, such specifications could have prevented real-world logic bugs [7], and our evaluation (Section 6) confirms their effectiveness in guiding correct synthesis across **XX** contracts.

Challenge 2: generalizability. Even with precise specifications, the space of implementations remains large. In particular, safety properties often under-constrain program behaviors, making the synthesizer produce overly restrictive contracts that rule out legitimate transaction sequences. Such contracts satisfy the specification but fail to generalize to unseen scenarios.

Maximally Permissive Synthesis. To generate safe yet general smart contracts, we adopt the notion of *maximally permissive synthesis*, which selects the implementation that admits the largest set of valid transaction traces among all correct candidates. A key challenge for maximally permissive synthesis is quantifying permissiveness. Since exact computation is infeasible, we approximate permissiveness using a trace-based method. In particular, we generate representative transaction traces and measure how many are accepted by a candidate contract. This approximation effectively guides the synthesis toward more general smart contracts.

Challenge 3: efficient synthesis under complex semantics. In general, it is challenging to synthesize a program over a large search space. Moreover, the rich semantics of real-world smart contracts introduce complicated constraints that can easily overwhelm state-of-the-art SMT solvers [8].

Combining deductive synthesis with trace-guided refinement. We address this challenge by dividing synthesis into two phases. Specifically, the first phase performs *deductive synthesis*, which generates a contract sketch using deterministic rules based on the given specifications [9]. The second phase conducts *inductive refinement*, which searches for completions that satisfy a set of concrete traces. To reduce the verification overhead, we precompute the contract state snapshots over these traces and avoid symbolically encoding full transitions. This approach narrows the search space and enables efficient constraint solving, offering a scalable alternative to syntax-guided synthesis [6, 29].

As a testbed, we implemented SmartSpec, a smart contract synthesizer based on a multi-modal specification that consists of the followings: (1) *functional specifications* in the form of inference rules

```

117 1 mapping(address => Stake) public stakes;
118 2 struct Stake { uint256 amount; bool unstaked; }
119 3 function unstake() external {
120 4     Stake storage s = stakes[msg.sender];
121 5     require(s.amount > 0); // Missing check:
122     require(!s.unstaked);
123 6     s.unstaked = true;
124 7     payable(msg.sender).transfer(s.amount);
125 8 }

```

Figure 1: Simplified staking example vulnerable to repeated withdrawals.

over contract states and transaction records; and (2) *safety specifications* expressed in temporal logic over transaction sequences. Leveraging both deductive and inductive synthesis techniques, SmartSpec generates an abstract representation of the smart contract that satisfies all specified properties while being maximally permissive. This abstract representation facilitates efficient verification against specified safety properties. Upon finding a satisfactory contract, its abstract representation is compiled into executable on the Ethereum [36] platform.

Contributions. Our main contributions are:

- **Multi-modal specification.** We formulate the smart contract synthesis problem using a specification framework that integrates functional specifications (inference rules) and safety specifications (temporal logic) (Section 4).
- **Synthesis algorithm.** We propose an efficient algorithm combining deductive and inductive synthesis with a compact encoding of contract semantics, together with soundness and relative completeness guarantees (Section 5).
- **Implementation and evaluation.** We implement SmartSpec and evaluate it on **XX** contracts, demonstrating effectiveness in synthesizing verifiably correct smart contracts (Section 6).

2 Background and Motivations

Smart contracts. Smart contracts are programs deployed on blockchains to manage digital assets and automatically enforce agreements without intermediaries. They encode core application logic such as token transfers, staking, and auctions, and execute deterministically once deployed. In practice, developers implement smart contracts in domain-specific languages such as Solidity [1] for Ethereum [36], as well as Vyper [4] and Move [37]. In this work, we focus on Solidity contracts due to its widespread adoption.

Vulnerabilities and specifications. We motivate the design of multi-modal specifications using two illustrative smart contract examples inspired by real-world incidents.

Example 1: Repeated withdrawal. Figure 1 shows a contract adapted from a real-world incident [15]. It allows users to stake and later withdraw their tokens. However, the unstake function fails to verify if a user has already withdrawn their stake. As a result, an attacker can repeatedly call unstake to drain the contract’s balance.

□

This vulnerability arises from contract-specific logic that is difficult for generic verification tools to capture [7, 8, 24, 26]. Even with a formal specification, existing tools only flag the issue post

```

1 function transfer(address sender, address
175     recipient, uint256 n) {
176     uint256 s = balance[sender];
177     uint256 r = balance[recipient];
178     require(s >= amount);
179     balance[sender] = s-n;
180     balance[recipient] = r+n; // vulnerable when
181     sender = recipient
182 }
183

```

Figure 2: Simplified transfer function vulnerable to inconsistent updates.

hoc. In contrast, our goal is to ensure such properties are *enforced by construction*, by automatically synthesizing guards that make the contract both provably correct and transparent in behavior.

Safety specification in temporal logic. The above vulnerability can be effectively captured by the specification that, for all stake index i , transaction $\text{unstake}(i)$ can happen at most once:

$$\Box(\text{unstake}(i) \rightarrow \neg\Diamond(\text{unstake}(i)))$$

Here, we follow the standard linear temporal logic syntax in Verx [26], where $\Diamond p$ denotes *once*(p), and $\Box p$ denotes *always*(p). With this specification, we ensure that every stake index can be unstaked at most once, thereby preventing the above attack and enhancing the contract’s security.

In addition to forbidden execution traces, another major class of vulnerabilities arises from incorrect state updates.

Example 2: Balance double-counting. The contract in Figure 2 is adopted from a real-world incident [14]. Suppose an attacker initially holds 20 tokens and transfers 10 tokens to its own address, *i.e.*, the sender and recipient are the same. The contract first reads the sender’s balance $s = 20$ and the recipient’s balance $r = 20$. It then subtracts 10 from s and adds 10 to r , resulting in a final balance of 30 tokens instead of 20. This inconsistency arises because both updates operate on the same account but are computed independently, violating the intended relational invariant that total tokens must remain constant. Such invariants are naturally expressed as inference rules. □

Functional specification in inference rules. This vulnerability can be prevented by enforcing an invariant stating that an account’s balance always equals the total tokens received minus the total tokens sent, ensuring that every balance reflects only legitimate transfers. See an example of such specification shortly in Section 3.

These examples demonstrate that both correctly validating incoming transactions and properly updating contract states are critical for maintaining smart contract safety. In the following sections, we show how SmartSpec uses these specifications to synthesize provably correct smart contracts.

3 Overview

Figure 3 shows the architecture of SmartSpec. It takes a multi-modal specification: (1) *inference rules* that define contract states in relational form over past transaction history, and (2) safety properties in *temporal logic* over transaction sequences.

The synthesis proceeds in two phases. First, it deductively transforms the inference rules into a contract sketch that encodes state updates without condition guards. Then, it completes the sketch

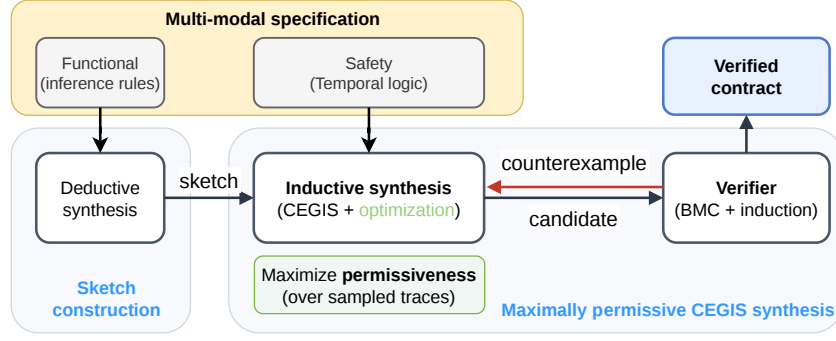


Figure 3: Overview of SmartSpec.

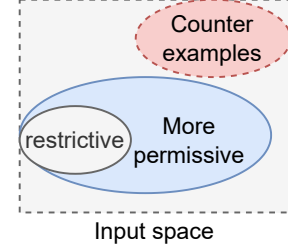


Figure 4: Permissiveness.

```

1 raised(s) :- s = sum m: invest(_, m).
2 investTotal(p,s) :- invest(p, _), s = sum m:
  invest(p,m).
3 refundTotal(p,s) :- refund(p, _), s = sum m:
  refund(p,m).
4 balanceOf(p,s) :- investTotal(p,i), refundTotal(p,
  r), s := i-r.

```

(a) Specification: inference rules over transaction records.

```

1 invest(p: address, n: uint256): {
2   raised <- raised + n;
3   balanceOf[p] <- balance[p] + n;
4 } // other transactions ...

```

(b) Partial program generated from inference rules, incrementally maintaining states on new transaction commits.

Figure 5: Phase 1: deductive synthesis.

by inferring transaction guards that ensure all safety properties are satisfied, using an iterative CounterExample-Guided Inductive Synthesis (CEGIS) procedure [30].

Inference rules. Figure 5a presents the inference rules for a crowdfunding contract, whose goal is to raise a target amount within a limited timeframe. Here, contract states are represented as relations, defined by inference rules [9] of the form *head* :- *body*. The rules read from right to left: if all predicates in the body hold, the head predicate is derived. The first rule defines an account's balance as the difference between its total incoming and spent tokens. For instance, Line 1 of Figure 5a sums all investments to compute the total raised. Lines 2–3 track each participant's invested and refunded amounts, and Line 4 derives their balance as the difference.

Step 1. Deductive synthesis (Section 5.1). Given the inference rules, SmartSpec deductively synthesizes a contract sketch as shown in Figure 5b, which incrementally maintains these states upon each transaction commit. For instance, the line `raised <- raised + n` in Figure 5b is generated from the rule in Line 1 of Figure 5a, where `raised` is defined as the sum of the amount column of the `invest` transactions. Therefore, when a new `invest` transaction is committed, `raised` is incremented by the value `n`.

Safety specification in temporal logic. Figure 6a illustrates three safety properties for the crowdfunding contract, expressed in temporal logic. For example, Line 4 specifies that whenever the crowdfunding is in the `OPEN` state, the sum of all participant balances must

```

1 // P1: In open state, balances sum to raised
  amount
2 .decl sumBalance(n: uint)
3 sumOfBalance(s) :- s = sum n: balanceOf(_,n).
4 □(state(s) ∧ sumOfBalance(m) ∧ raised(n) ∧ s!=
  OPEN ⇒ m>n));
5
6 // P2: Refund subtracts balance from raised
  amount
7 □(refund(p,n) ∧ •(balanceOf(p,n')) ∧ raised(m)
  ∧ •(raised(m'))
  ⇒ m==m'-n');
8
9
10 // P3 Withdraw and refund cannot be both
  executed
11 □(¬(♦(withdraw) ∧ ♦(refund)));

```

(a) Specification in linear temporal logic.

```

1 invest(p,n) :- msgSender(p),msgValue(n),closed
  (false),raised(s),target(t),s<t.
2 closed(true) :- msgSender(s), owner(s).
3 refund(p,n) :- msgSender(p),closed(true),
  raised(r),target(t),r<t, balanceOf(p,n),n
  >0.
4 withdraw(p,r):- msgSender(p), beneficiary(p),
  raised(r),target(t),r>=t.

```

(b) Synthesis intermediary output: transaction validation rules.

Figure 6: Phase2: inductive synthesis.

equal the raised amount. The operator $\Box$ (always) enforces that the specified invariant must hold throughout all reachable states. Other operators, such as $\bullet$ (prev, Line 7) and $\blacklozenge$ (once, Line 11), refer to the preceding and any past transaction states, respectively.

Step 2. Counterexample-guided inductive synthesis (Section 5.2). The goal of this step is to infer transaction guards so that the contract satisfies the safety properties. Because temporal safety often spans multiple transactions, it cannot be encoded by local deductive rules alone. SmartSpec therefore adopts a CEGIS loop. Specifically, SmartSpec starts with empty guards and checks the sketch against the verifier. If there is a safety violation, the verifier returns a counterexample trace, from which SmartSpec infers new guards to block the unsafe behavior. This loop continues until a fully-verified contract is found or it reaches the iteration bound.

```

Decl ::= RelDecl*
RelDecl ::= SingletonR( $\bar{T}$ ) | Tx R( $\bar{T}$ ) | Rel R( $\bar{T}$ ) [Key]
Key ::= [ $i_1, i_2, \dots$ ]
T ::= int | addr | bool | ...

```

(a) Relation declarations.

```

Rule ::= H( $\bar{x}$ ) :- Body
Body ::= Join | R( $\bar{x}$ ), y = Agg : R( $\bar{y}$ )
Join ::= R( $\bar{x}$ ) | Pred, Join
Agg ::= sum n | max n | min n | count
Pred ::= R( $\bar{x}$ ) | C( $\bar{x}$ ) | y = F( $\bar{x}$ ) | y = UDF( $\bar{x}$ )
UDF ::= id( $\bar{x}$ )
C ::= > | < | ≥ | ≤ | ≠ | ==
F ::= + | - | × | /

```

(b) Inference rules.

```

F ::= atomic | u_op F | F bin_op F
atomic ::= R( $\bar{x}$ ) | C( $\bar{x}$ )
u_op ::= ¬ | ♦(once) | □(always) | •(prev)
bin_op ::= ∧ | ∨ | →

```

(c) Temporal logic for safety specification.

Figure 7: SmartSpec specification syntax.

Disambiguate via maximal permissiveness (Section 5.3). In practice, even comprehensive safety properties and inference rules rarely capture a contract in full, leaving room for multiple correct implementations. Simple criteria like minimal code size may produce trivial but useless contracts, for instance, one that always rejects unstake calls is technically safe but defeats the contract's purpose. To avoid this, SmartSpec adopts a maximally permissive strategy. Fully quantifying permissiveness is intractable because the trace space is infinite. Instead, SmartSpec approximates it by sampling legal traces from the safety specification.

As illustrated in Figure 4, the input space includes both safe and unsafe regions: the top-right area denotes counterexamples, while two alternative candidates are safe but differ in permissiveness, and the one admitting more valid traces is preferred.

Scope and limitations. SmartSpec focuses on contracts whose logic can be expressed as inference rules with deterministic guards and fixed, side-effect-free user-defined functions (UDFs), and does not handle external interactions (e.g., oracle calls or cross-contract calls). Safety guarantees follow the verifier (unbounded with inductive invariants, otherwise bounded), and maximal permissiveness is defined with respect to sampled traces, thus approximating the full behavior space.

4 Problem Formulation

This section formalizes the synthesis problem. We begin by defining the syntax of inference rules and safety properties, then specify the output program space.

Functional specification. We start by introducing relations, the basic building blocks of contract states. We then define predicates constructed over these relations, and finally describe how rules are formed from predicates to specify contract functionality.

Relations. Contract states are modeled as relations declared by the user (Figure 7a) and defined through inference rules. We support three kinds of declared relations: (1) *Singleton relations*, which contain exactly one tuple and represent global configuration parameters (e.g., funding target or deadline); (2) *Transaction relations*, which record transaction invocations such as invest or refund; and (3) *Simple relations*, which store persistent contract state and may specify primary key columns [$i_1, i_2, \dots$] to uniquely identify tuples (e.g., balanceOf(addr, amount) keyed by address). Platform-provided primitives (e.g., msg.sender, timestamp, send) are built-ins and need not be declared.

Predicates. As shown in Figure 7b, predicates (*Pred*) are the logical atoms used in rule bodies and transaction guards. They include: (1) relational atoms $R(\bar{x})$ over declared relations, (2) comparisons $C(\bar{x})$ between domain-consistent values, (3) arithmetic function applications $y = F(\bar{x})$, and (4) calls to user-defined functions (UDFs).

UDFs allow the specification to incorporate deterministic computations that are difficult to express using only relational joins and basic arithmetic. Typical use cases include cryptographic hash checks, signature-related logic, and custom financial formulas (e.g., interest or dividend calculations in DeFi). Syntactically, a UDF appears in rule bodies as a predicate of the form $y = f(\bar{x})$ (Figure 7b), evaluated as a pure function call rather than materialized as a database relation. In this work, UDFs are treated as given, side-effect-free functions with fixed semantics, and their outputs are deterministically evaluated on traces during both synthesis and verification.

Example 3: User-defined functions. Consider a staking contract with a custom reward function $\text{reward}(p, t)$ that computes accrued interest for participant p at time t :

$$\text{reward}(p, t) = r \cdot (t - \text{stakeTime}(p))$$

where r is a fixed interest rate and $\text{stakeTime}(p)$ records when participant p staked. It can appear in predicates and transaction guards, e.g., $\text{reward}(p, t, r)$, $r > 0$. Similarly, UDFs can capture hash checks or DeFi-specific formulas. □

Rules. SmartSpec uses Horn-clause style inference rules, a subset of first-order relational logic. As shown in Figure 7b, each rule has the form $\text{Head}(\bar{x}) :- \text{Body}$, read right to left: if the body holds, the head tuple is derived and inserted into its relation. We distinguish two rule types.

(1) *Join rules.* The body of a rule may be a conjunction of predicates, forming a Join rule. Such a rule holds if there exists a variable assignment $\pi : V \mapsto D$ that makes all literals true. Relational literals are satisfied when the corresponding tuple exists in the relation, while conditions and functions impose constraints on variables. For example, the rule at Line 4 in Figure 5a joins $\text{investTotal}(p, i)$ and $\text{refundTotal}(p, r)$ on the participant p , and derives $\text{balanceOf}(p, s)$ with $s = i - r$.

(2) *Aggregation rules.* These take the form $y = \text{Agg } n : R(\bar{y})$, which computes an aggregate over all rows in $R(\bar{y})$ matching a given variable assignment.

```
investTotal(p, s) :- invest(p, _), s = sum m : invest(p, m).
```

This rule groups the invest table by the first column (p) and, for each group, sums the second column (m) to produce investTotal.

Feature	Restriction
ETH transfer	Supported.
Loops	Bounded loops.
Recursion	No recursion.
External calls	No calls to external contracts.
Contract	No contract creation or destruction.

Figure 8: Fragments of Solidity supported by SmartSpec.

A rule is *well-formed* if all variables are grounded, *i.e.*, each variable must appear either as a transaction parameter or as a bound variable in a relational atom. For example, in `transfer(sender, recipient, amount)`, predicates such as `amount > 0` or `balanceOf(sender, b), b >= amount` are well-formed, whereas `x > 0` is not if `x` appears nowhere else.

Safety specification. Figure 7c shows the syntax of past time linear temporal logic [26]. Atomic propositions represent relations $R(\bar{x})$ and conditions $C(\bar{x})$, whose syntax are defined in Figure 7b. Temporal operators include *once*, *always*, and *prev*. In addition, formulas can be constructed through logical connectives including conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$). There are two typical approaches to specify safety properties:

(a) Patterns over transactions. These are constructed using temporal and logical operators over transactions. For instance:

$$\Box(\neg(\Diamond(\text{withdraw}) \wedge \Diamond(\text{refund})))$$

where *withdraw* and *refund* represent two types of transactions. This formula indicates that once a crowdfunding beneficiary has withdrawn funds, no investor can request a refund, and vice versa.

(b) Patterns over contract states. Relational predicates $R(\bar{x})$ are instantiated on relations representing contract states. For instance:

$$\Box(\text{state}(s) \wedge \text{sumOfBalance}(m) \wedge \text{raised}(n) \wedge s \neq \text{OPEN} \rightarrow m \geq n)$$

This formula specifies an invariant over contract states, represented by relation *state*, *sumOfBalance*, and *raised*. This invariant states that, if the contract state is not *OPEN*, the total balances of all participants must be greater or equal to the raised amount.

Output space. SmartSpec targets a subset of Solidity, using DeCon [9] as an intermediate representation. As shown in Figure 8, this subset supports ETH transfers and bounded loops over arrays, but excludes recursion, external calls, and contract creation or destruction, since the verifier [8] cannot handle them. Recursion is disallowed in the specification to ensure termination and is uncommon in practice [9]. Extending SmartSpec to reason about external interactions remains important future work.

5 Synthesis Algorithm

This section presents the deductive and inductive synthesis algorithm, and synthesis output disambiguation via maximal permissiveness. We build on existing verification infrastructure [8]: bounded model checking for counterexamples, induction for proofs, and declarative structure for efficient invariant search. Details in the full version [3].

5.1 Deductive Synthesis

In the deductive synthesis phase, SmartSpec repurposes the transformation rules of DeCon [9], a declarative language for smart

Algorithm 1 CEGIS Loop with Maximal Permissiveness.

Input: (1) Φ : safety specification; (2) $\mathcal{R}$: functional rules; and (3) K : iteration bound (hyperparameter).
Output: A synthesized contract C or $\perp$ if not found.

```

1:  $\mathcal{T}_{\text{dis}} \leftarrow \text{SAMPLEVALIDTRACES}(\Phi)$   $\triangleright$  Disambiguation (Sec. 5.3).
2:  $\mathcal{P} \leftarrow \text{ENUMERATEPREDICATES}(\mathcal{R})$ 
3:  $\mathcal{T}_{\text{cex}} \leftarrow \emptyset$ 
4:  $C \leftarrow \text{INITSKETCH}(\mathcal{R})$   $\triangleright$  Section 5.1.
5:  $\text{iteration} \leftarrow 0$ 
6: while  $\text{iteration} < K$  do
7:    $\text{iteration} \leftarrow \text{iteration} + 1$ 
8:    $\tau \leftarrow \text{BMC}(C, \Phi)$   $\triangleright$  Find counterexample trace
9:   if  $\tau = \perp$  then
10:    return  $C$   $\triangleright$  No counterexample, synthesis complete
11:   else
12:     $\mathcal{T}_{\text{cex}} \leftarrow \mathcal{T}_{\text{cex}} \cup \{\tau\}$ 
13:   end if
14:    $C \leftarrow \text{BLOCKTRACECONSTRAINTS}(\mathcal{T}_{\text{cex}}, \mathcal{P})$   $\triangleright$  Section 5.2.
15:    $J \leftarrow \text{PERMISSIVENESSOBJECTIVE}(\mathcal{T}_{\text{dis}}, C)$   $\triangleright$  Section 5.3.
16:    $C \leftarrow \text{OPTIMIZE}(C, J)$   $\triangleright$  Z3: satisfy constraints, maximize permissiveness
17: end while
18: return  $\perp$   $\triangleright$  Iteration bound reached

```

contracts, to generate executable code from inference rules. Unlike DeCon, which requires developers to manually specify conditions, SmartSpec focuses only on generating state maintenance logics, delegating condition generation to an automated synthesizer.

For each transaction type, the contract sketch consists of the state maintenance instructions for newly committed transactions. The deductive synthesis begins by identifying dependencies among declared relations. A relation R depends on Tx if there exists a rule where R is in the head and Tx in the body. Starting with the relation for Tx , update instructions are derived to ensure the validity of the inference rule results as new relations are derived. The dependency relation is transitive: updates to a relation trigger further updates to other affected relations. This process iterates through the dependency chain until no further updates are needed. Since recursive relations are prohibited in the specification (Section 3), this process is guaranteed to terminate.

Example 4: Figure 5b shows the contract sketch for the crowdfunding example. The deductive phase produces update instructions along dependency chains. In this example, instructions are generated following two dependency chains: `invest` $\rightarrow$ `raised`, and `invest` $\rightarrow$ `investTotal` $\rightarrow$ `balanceOf`. Note that `investTotal` is eliminated by the compiler optimizer that short-circuits updates from `invest` to `balanceOf`. $\square$

5.2 Inductive Synthesis

Building on the contract sketch generated above, SmartSpec refines it by inferring transaction guards that ensure safety. The goal is to rule out all counterexample traces produced by bounded model checking while avoiding overly restrictive conditions. To this end, SmartSpec adopts a constraint-based, counterexample-guided inductive synthesis (CEGIS) procedure enhanced with a maximal

permissiveness objective, which favors solutions that are as general as possible. The rationale and formulation of this objective are detailed in Section 5.3.

Overview. Algorithm 1 outlines the CEGIS loop. Given functional rules $\mathcal{R}$ and safety specification Φ , SmartSpec first samples a set of disambiguating traces $\mathcal{T}_{\text{dis}}$ that satisfy Φ (Section 5.3) and constructs the predicate set $\mathcal{P}$ by enumerating type-consistent bindings of relational variables and comparison operators (Figure 7b) for guard synthesis; an optional optimization restricts the binding space using domain conventions (Section 5.2). Next, a contract sketch C is initialized from $\mathcal{R}$ (Section 5.1), containing state updates but no guards.

In each iteration, bounded model checking (BMC) verifies C against Φ . If no counterexample is found, synthesis terminates. Otherwise, the counterexample τ is added to $\mathcal{T}_{\text{cex}}$, and SmartSpec constructs blocking constraints C from $\mathcal{T}_{\text{cex}}$ and $\mathcal{P}$ (Section 5.2) and defines a permissiveness objective J that measures the number of disambiguating traces accepted by C . The solver Z3 optimizes C to satisfy C while maximizing J , repeating until no counterexample is found or the iteration bound K is reached.

Predicate generation (line 2). For each transaction type Tx , SmartSpec constructs a predicate set $P(Tx)$ used in guard synthesis; $\mathcal{P}$ aggregates these sets. Predicates are formed by instantiating domain-consistent comparison operators (C) of Figure 7b. Variables in a predicate may originate from either the transaction parameters or contract states represented as relations.

For example, in a transaction `transfer(sender, recipient, amount)`, predicates may include parameter-based conditions such as `amount > 0`, or state-dependent ones such as `balance(sender) >= amount`, where variables are bound to relational states.

A predicate is considered *well-formed* if all of its variables are grounded, *i.e.*, each variable must appear either as a transaction parameter or as a bound variable in a relational tuple. For example, in `transfer(sender, recipient, amount)`, the predicate `balance(sender) >= amount` is well-formed because both `sender` and `amount` are grounded. In contrast, `balance(p) >= amount` is ill-formed if `p` is not bound to any parameter or relational variable, since its value cannot be resolved during evaluation. This ensures that all predicates can be evaluated concretely over the current contract state and transaction inputs.

Optimization via parameter-pattern restriction. Full enumeration yields a Cartesian explosion and incorrect bindings. We apply simple heuristics (*e.g.*, distinct roles bind to distinct parameters; non-zero for amounts and addresses) and restrict to these patterns first, falling back to the full space if UNSAT. For `transferFrom`, `allowance(owner, spender, m)` is correct; `allowance(recipient, spender, m)` is not. Completeness is preserved: Proposition 5 holds regardless of which search phase succeeds.

With the predicate set $\mathcal{P}$ in place, we next describe the contract encoding and constraint generation process (Algorithm 1, line 12).

Internal contract representation. Building on the predicate set generated above, SmartSpec encodes each candidate transaction guard into a constraint representation suitable for solver optimization. As shown in Figure 9, each conjunctive clause is represented as a one-hot vector over the predicate space. For example, the vector `100010...` corresponds to the conjunction of the first and fifth

predicates, `owner == sender ∧ to ≠ 0x0`. This encoding bridges the high-level logical specification and the bit-vector formulation used by the solver in Algorithm 1.

The entire contract is represented as a mapping $\Phi : Tx \rightarrow \text{List}[\text{Boolean}]$, where each transaction type Tx is associated with a Boolean vector of length $|P|$ (the size of the predicate set), indicating the inclusion of each predicate in its guard.

Constraints on example traces. To ensure correctness, we require that viable candidate contracts reject all counterexamples. Rather than symbolically encode the full contract semantics, we simplify constraint generation by evaluating predicate truth values on concrete traces and collecting the results in a lookup table $\mathcal{D}$ over which the constraints are defined.

(a) Evaluate predicate truth value. Using the contract sketch, which specifies how state updates occur upon transaction commits, we simulate the execution of concrete example traces to precompute the contract state at each step. This allows us to directly evaluate the truth value of each predicate $P(Tx)$ for every transaction Tx in the trace, bypassing symbolic execution and simplifying the encoding.

Example 5: Table 1 shows predicate truth values for an example trace. Each row corresponds to the state after committing a transaction (header column), with predicate values computed from the evolving contract state based on the partial sketch. For instance, `ended` is initially false and becomes true after the end transaction. Predicates irrelevant to a transaction type (*e.g.*, `n > 0` for `end` or `withdraw`) are marked NA. $\square$

(b) Encoding example traces as constraints. Each example trace is encoded as constraints over its predicate truth table: a transaction is approved if all predicates in its guard is satisfied. Formally, let Γ_T be the truth table for trace T :

$$\text{approve}(tx, \Phi) \triangleq \forall i. \Phi[i] \rightarrow \Gamma_T[tx][i]$$

Here, $\Phi[i]$ indicates whether the i -th predicate is included in the guard of transaction tx . The implication enforces that if a predicate is selected in the guard, it must evaluate to true for the transaction in the given trace.

A trace is considered valid only if all its transactions are approved:

$$\text{approve}(T, \Phi) \triangleq \forall tx \in T. \text{approve}(tx, \Phi)$$

Intuitively, this means that for a trace to be accepted, every transaction within it must satisfy all predicates selected in its guard.

For a counterexample trace set Neg , the constraint is:

$$\{\neg \text{approve}(T, \Phi) \mid T \in \text{Neg}\}$$

Together, these encodings define the constraint space used by the inductive synthesizer. Each counterexample trace contributes a blocking constraint that eliminates unsafe behaviors, while satisfying assignments to Φ represent candidate guard conditions consistent with all collected examples. In the subsequent optimization phase, SmartSpec searches for the most permissive solution that satisfies these constraints, as described in Section 5.3.

5.3 Maximally Permissive Synthesis

As discussed in Section 3, the admissible behaviors of a smart contract are defined by temporal safety properties. However, contracts that reject most or all transactions, though safe, are useless. To

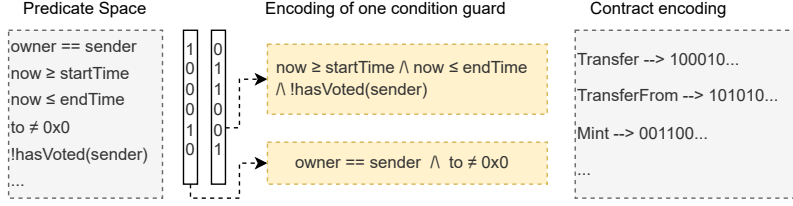


Figure 9: Encoding of a candidate contract

avoid such outcomes, SmartSpec adopts a maximally permissive synthesis strategy, aiming to accept as many valid traces as possible without violating safety properties.

Since the trace space is infinite, exact quantification is infeasible. We instead sample a subset of valid traces and maximize the number accepted. A small but diverse sample is often sufficient to distinguish generalizable contracts from overly restrictive ones.

Trace sampling. We construct a small set of up to N blockchain addresses and randomly initialize parameters such as the contract owner and permitted users. Setup transactions are executed to populate account states, since many checks depend on balances or role assignments (e.g., owner, operator). For each transaction type, we randomly draw type-consistent inputs, execute them to generate traces, and retain only those satisfying safety properties. For efficiency, a fixed number of traces (N per transaction type) is sampled.

The permissiveness of each candidate contract is encoded as an optimization objective in the Z3 solver used during the inductive phase. The solver thus searches for contracts that block all counterexample traces while accepting as many valid traces as possible.

Example 6:[Permissiveness disambiguation] Consider a staking contract with the safety property that each stake index i can be unstaked at most once. Suppose the sampled valid trace set contains:

- $$\begin{aligned} T_1 &: \langle \text{stake}(a, 10, 7), \text{advance}(8), \text{unstake}(a, i) \rangle \\ T_2 &: \langle \text{stake}(b, 20, 30), \text{advance}(31), \text{unstake}(b, j) \rangle \\ T_3 &: \langle \text{stake}(a, 5, 7), \text{advance}(8), \text{unstake}(a, k) \rangle \end{aligned}$$

and a counterexample trace:

$$T_x : \langle \text{stake}(a, 10, 7), \text{advance}(8), \text{unstake}(a, i), \text{unstake}(a, i) \rangle.$$

Consider two candidate guards for $\text{unstake}(u, i)$:

$$C_1 : \text{lockExpired}(i) \wedge \neg \text{unstaked}(i) \wedge (\text{msg.sender} = \text{owner})$$

$$C_2 : \text{lockExpired}(i) \wedge \neg \text{unstaked}(i)$$

Both candidates block T_x and satisfy the safety property. However, C_1 rejects T_1, T_2, T_3 unless the owner performs every withdrawal, while C_2 accepts all three valid traces by allowing the stake owner to withdraw after expiry. Hence C_2 is more permissive on the sampled traces and is preferred by SmartSpec. $\square$

Remarks. Maximal permissiveness is guaranteed only with respect to the generated traces; more permissive correct implementations may exist if the trace set is biased. In practice (Section 6), we find that 50 traces per transaction type suffice to avoid such cases.

5.4 Properties

We formalize the correctness guarantees of SmartSpec. Specifically, we establish (1) soundness with respect to the functional and safety

Table 1: Truth table for an example trace.

Transactions	ended	sender==owner	n>0	...
invest(a1,10)	F	F	T	
invest(a2,50)	F	F	T	
invest(a3,60)	F	F	T	
end()	T	T	NA	
withdraw(a1)	T	F	NA	

specifications, and (2) relative completeness within the defined search space.

We first state a lemma about the verifier, which forms the foundation of SmartSpec's soundness. **Lemma 1 (Verifier guarantee):**

Let Φ be a candidate contract.

(1) If the verifier proves Φ as an inductive invariant, then Φ holds for traces of arbitrary length.

(2) If induction fails but no counterexample of length $\leq B$ is found, then Φ holds for all traces of length at most B . $\square$

Proofs are deferred to Appendix B.

Proposition 2 (Soundness of inductive phase): Let C_0 be a contract sketch and Φ a safety specification. If the inductive phase of SmartSpec returns a contract C , then C satisfies Φ under the guarantees provided by the verifier (Lemma 1). $\square$

Proof sketch. SmartSpec returns a contract C only if the verifier reports no violation of Φ . By Lemma 1, this implies that C satisfies Φ under the verifier's guarantees. $\square$

Lemma 3 (Soundness of deductive phase): Let $\mathcal{R}$ be a well-formed set of functional inference rules. Let $C_0 \leftarrow \text{INITSKETCH}(\mathcal{R})$ be the contract sketch generated by the deductive phase. Then for every transaction sequence T , the states produced by executing C_0 correspond to the tuples derivable from $\mathcal{R}$ under its relational semantics. $\square$

Based on the above lemmas, we establish soundness of SmartSpec.

Proposition 4 (Soundness): Let $\mathcal{R}$ be the functional specification, Φ the safety specification, and T the sampled trace set used to approximate permissiveness. If SmartSpec returns a contract C , then:

- (1) C satisfies the functional specification $\mathcal{R}$;
- (2) C satisfies the safety specification Φ ;
- (3) among all contracts satisfying (1) and (2) within the search space, C maximizes the number of traces in T that are admitted. $\square$

In other words, the deductive phase enforces the intended state semantics, while the inductive phase synthesizes guards that ensure safety. Among all safe completions of the sketch, SmartSpec returns one that maximizes permissiveness over the sampled traces T .

Proof sketch. By Lemma 3, the sketch preserves the functional semantics of $\mathcal{R}$, and the inductive phase only adds guards without modifying state updates, so the final contract C still satisfies $\mathcal{R}$. Since SmartSpec returns C only after the verifier reports no violation of Φ , Lemma 1 ensures that C satisfies Φ under the verifier guarantee. Moreover, as permissiveness over the sampled trace set T is encoded as an optimization objective, the solver returns a model maximizing it among all satisfying assignments. Hence C satisfies $\mathcal{R}$ and Φ and is maximally permissive with respect to T within the search space. $\square$

Definition 1:[Guard search space] Let P be the set of predicates generated from declared relations, domain-consistent comparisons, arithmetic expressions, and user-defined functions (UDFs); SmartSpec may first search a domain-convention-restricted subset and fall back to the full set if the restricted search fails (Section 5.2).

The guard search space $\mathcal{H}$ consists of all guards that can be constructed as conjunctions of predicates from P . $\square$

Proposition 5 (Relative completeness): Let $\mathcal{R}$ be the functional specification and Φ the safety specification. Let $\mathcal{H}$ be the guard search space defined in Definition 1.

If there exists a contract C^* obtained by instantiating the sketch with guards from $\mathcal{H}$ such that C^* satisfies Φ , then SmartSpec will return some contract $C \in \mathcal{H}$ that satisfies Φ . $\square$

Proof sketch. The inductive phase searches the finite hypothesis space $\mathcal{H}$ via constraint solving. Assuming the underlying solver is complete for the encoded constraint theory, if a satisfying guard assignment exists within $\mathcal{H}$, the solver can discover one. Hence SmartSpec returns such a contract after successful verification. $\square$

Proposition 6 (Complexity): Let $|\mathcal{P}|$ be the number of predicates, m the number of counterexample traces, and L the maximum trace length. In each CEGIS iteration, SmartSpec constructs constraints of size $O(m \cdot L \cdot |\mathcal{P}|)$ and invokes the SMT solver once. Thus, the encoding cost per iteration is polynomial in m , L , and $|\mathcal{P}|$, and the total runtime scales linearly with the number of iterations. $\square$

The dominant cost of SmartSpec lies in the CEGIS loop. Each iteration involves: (1) bounded model checking to search for counterexamples, and (2) solving a constraint optimization problem whose size is proportional to the number of predicates and accumulated traces. In practice, BMC is the main time consumer, since it explores execution paths over the contract state space, while constraint solving operates over a compact bit-vector encoding of predicates. Predicate truth evaluation and trace simulation are comparatively lightweight and parallelizable across traces.

This explains two empirical observations: (i) synthesis time grows with the number of relations and transaction types, because they enlarge the predicate space and the transition structure explored by BMC; (ii) per-iteration time remains relatively stable across benchmarks, since the solver operates over similarly structured constraint templates. Thus, performance is primarily governed by the complexity of the state space exposed to the verifier, rather than by the inductive encoding itself.

6 Evaluation

We aim to answer the following research questions:

- (1) **Effectiveness and efficiency.** Can SmartSpec synthesize different contracts correctly and efficiently?
- (2) **Output quality.** Can SmartSpec generate efficient smart contracts compared to their hand-written counterparts?
- (3) **Improvement over baseline tools.** What are the differences and advantages of SmartSpec compared to existing tools?

Benchmarks. We evaluate SmartSpec on 31 smart contracts and their temporal specifications from prior work on smart contract verification [8, 26]. We include only contracts that satisfy two criteria:

- (1) no essential cross-contract interactions (calls, proxies, or delegates), because SmartSpec synthesizes standalone single contracts; and
- (2) core logic expressible as relational logic, optionally via UDFs. Excluded are, for instance, contracts using recursive data structures, Merkle trees, or zero-knowledge proof verification. For the selected contracts we perform lightweight normalization by extracting core logic into relational rules and encoding contract-specific formulas as fixed side-effect-free UDFs; incomplete specifications are supplemented with standard safety properties from documentation. Table 2 shows the benchmark characteristics. Full benchmarks are in the SmartSpec artifact [3].

Experiment setup. SmartSpec is implemented in 2k lines of Scala, extending DeCon [9] for deductive synthesis and Solidity support, integrating DCV [8] for temporal verification, and using Z3 [13] v4.8.14 for constraint solving. Bounded model checking explores up to 5 transactions per search. For permissiveness analysis, we sample 50 traces per transaction type. We set an hour timeout. All experiments run on a 4 GHz CPU with 24 GB memory.

6.1 Effectiveness and efficiency

Effectiveness. As shown in Table 2, SmartSpec successfully synthesizes XX% of benchmark contracts in the initial run. All synthesized contracts pass property verification and are manually validated to preserve equivalent logic with their reference implementations. For failure cases where the initial run yields an incorrect solution, we apply a feedback mechanism (details below) and report a separate success rate within a budget of 10 iterations; final results are summarized in the table.

Failure cases. We observe two types of failures.

(1) *Limitation of permissiveness approximation.* Some contracts are overly restrictive because the constraint solver breaks ties arbitrarily when several candidates look equally permissive. We approximate permissiveness by counting accepted traces on randomly sampled sets, so the approximation is inexact and cannot distinguish a strictly more permissive guard from a stronger one; both can tie and the solver may return the wrong one. In **vestingWallet**, the `release()` guard keeps a redundant predicate `released  $\neq$  0` and is stricter than necessary.

(2) *Spurious predicate bindings.* A guard is spurious when it includes predicates irrelevant to the intended precondition—e.g., a transfer guard incorrectly requiring `totalSupply` or another balance. Guards are conjunctions over a finite predicate set (Section 5.2); the solver only enforces consistency with collected positive and counterexample traces, so predicates accidentally true on the current trace set can be included.

Handling of failure cases. To address wrong guards from the two failure modes above, we introduce a simple interactive step: the user forbids the offending predicate combination (e.g., a guard for `increaseAllowance()` that wrongly conjoins `balanceOf[spender] > 0`), and the synthesizer re-runs with that combination disabled; we repeat as needed. Under the 10-round cap, success reaches XX%. These limitations reflect the fundamental difficulty of precise specification; the lightweight feedback nevertheless recovers close to full coverage.

Table 2: Benchmark characteristics and synthesis results, with number of relations (#rel.), transactions (#tx.), rules (#rules), safety properties (#prop.), UDF count (#UDF); Success: Y = acceptable, N→Y = repaired after interactive forbid-based feedback (≤ 5 rounds), N = residual failure, N→N = still fail; synthesis time and iterations; and gas usage compared with baselines.

Benchmark	LoC	Specification					Success Result	Synthesis Time		Gas Efficiency	
		#rel.	#tx.	#rules	#prop.	#UDF		time(s)	#iter.	Gas(K)	Diff
auktion	156	16	4	15	10	0	N→Y	0.16	0	56	-2.4%
bnb	265	25	11	15	13	1	Y	1.29	3	48	+5.8%
brickBlockToken	293	23	13	19	10	0	N→N	0.28	1	44	-2.8%
cappedCrowdSale	321	22	15	13	26	1	Y	0.32	0	63	-1.7%
controllable	275	18	12	14	29	0	Y	0.37	2	49	+0.6%
crowdFunding	188	14	8	13	6	0	Y	0.38	3	59	+1.9%
crowdsale	447	37	15	32	31	1	Y	0.75	2	94	-1.4%
crowdsale2	454	37	15	32	37	1	Y	0.25	0	94	-1.1%
erc1155	201	18	8	11	14	1	N	1.49	4	27	+7.5%
erc20	206	17	8	13	19	0	N→Y	0.16	1	48	+1.4%
erc777	320	21	15	14	37	1	Y	1.22	3	48	+1.6%
finalizableCrowdSale	258	23	10	13	24	1	Y	0.58	2	49	+0.4%
linktoken	249	20	9	17	13	0	N→Y	0.19	1	54	+2.5%
ltxSwapAsset	281	23	9	18	20	0	N→Y	0.11	0	30	+0.7%
mana	184	23	4	25	14	0	Y	0.26	0	49	+4.0%
matic	285	21	13	14	18	0	Y	0.53	3	44	+4.9%
nft	304	5	11	11	16	1	Y	0.29	0	49	-2.3%
paymentSplitter	118	10	3	11	5	0	Y	0.09	1	45	+0.0%
shib	230	17	9	13	12	0	N→Y	0.20	1	45	-4.3%
stbt	270	23	9	18	24	2	Y	0.85	2	48	+0.9%
tether	252	20	11	12	13	1	Y	0.23	0	48	+1.1%
theta	250	22	10	16	20	1	N→Y	1.22	3	53	-3.8%
tokenPartition	167	14	6	10	12	0	Y	0.83	3	66	+0.1%
vestingWallet	108	10	2	7	4	1	Y	0.16	0	32	+10.0%
voting	155	6	8	4	6	0	Y	0.15	2	74	+3.2%
wallet	134	10	5	7	12	0	Y	0.11	1	47	+2.9%
wbtc	313	22	13	15	19	0	N→Y	0.25	1	51	-2.4%
jokintheboxstakin	209	13	12	7	7	3	Y	1.67	2	108	-3.2%
level	164	16	9	10	8	1	Y	0.35	0	46	-7.1%
metadragon	182	17	8	10	8	2	N→Y	0.58	2	30	-7.3%
sheepFarm	225	16	12	9	5	3	Y	1.01	1	150	-3.0%
Average / Total							64.5%→93.5%				

Efficiency. The total synthesis time ranges from 4s to **XX** and is linearly correlated with the number of CEGIS iterations, with an average per-iteration time of about **XX** across all benchmarks. Synthesis time also increases with the number of relations (#rel.), as more relations enlarge the program search space, and the predicate space grows combinatorially with the relation space (Section 5.2). Contracts with more transaction types also require longer synthesis time, since additional rules must be derived per transaction type.

Overhead analysis. In practice, the verifier dominates the per-iteration cost, while constraint solving over the guard variables is comparatively lightweight due to the one-hot encoding of predicates. As a result, contracts with more complex temporal properties incur higher cost than those with larger state schemas. Nevertheless, synthesis remains within minutes for all benchmarks, suggesting that the CEGIS loop scales well for medium-sized smart contracts.

Safety verification. Safety properties over shared states are the most time-consuming, since many transaction types update these states and each guard requires more predicates to be filled, leading to more CEGIS iterations. For example, in tether, both $\square(\forall p. \text{balance}[p] \geq$

0) and $\square(\forall p, q. \text{allowance}[p][q] \geq 0)$ must hold. Because all transaction types (issue, transfer, etc.) update one or both of these states, each guard must incorporate predicates that preserve the invariants.

6.2 Output quality

We compare the efficiency of synthesized contracts with handwritten references from open repositories, using gas consumption [36] as the metric. Gas is measured by deploying each contract on a local Truffle [2] blockchain. To ensure functional equivalence, reference contracts using unsupported features (Figure 8) are simplified. We then generate random test workloads to simulate various transaction scenarios.

SmartSpec achieves comparable gas efficiency with the handwritten reference implementation. The gas consumption measurements are presented in the columns under the "Gas consumption" group in Table 2. The gas difference percentages range from -5.9% to +5.3%, with an average at -0.6%.

Gas Overhead and Optimizations. Overhead primarily arises from auxiliary variables introduced by SmartSpec. For example,

to support relational representations of contract states, SmartSpec encodes each relation as a Tuple. Even a simple mapping such as `balanceOf` is represented as a mapping from an address to a tuple containing both the value and a validity bit. The validity bit, introduced by DSC, distinguishes records invalidated by newer derivation results. However, such overhead can be mitigated by borrowing standard optimization techniques from general-purpose compiler design for declarative programs, which is an important future work.

Conversely, some contracts achieve gas savings through two optimizations: (1) SmartSpec consolidates related modules into a single contract, reducing redundancy and external call overhead (e.g., integrating Escrow and Crowdsale in crowdfunding); (2) it prioritizes cheap checks (e.g., `amount > 0`) before costly state-dependent ones (e.g., `balance[sender] >= amount`), avoiding unnecessary computations and storage accesses.

Maximal permissiveness is critical for generalizability. Without the maximal permissiveness objective, SmartSpec often converges to trivially restrictive implementations, e.g., enforcing both $n > 0$ and $n \leq 0$, which reject all transactions and are thus vacuously safe but functionally useless. This demonstrates that safety properties alone provide insufficient guidance for synthesis: many “correct” programs exist, but most fail to capture intended functionality. Incorporating maximal permissiveness biases the search toward safe implementations while preserving usability, aligning synthesized behavior more closely with real-world contract logic.

6.3 Qualitative comparison with baseline tools

To the best of our knowledge, no existing synthesis tool that synthesizes smart contracts from the combination of examples, relational logic and temporal logic. We show a qualitative comparison with the two close related tools.

State-machine synthesis. SCSYNT [17] synthesizes smart contracts from TSL specifications, an LTL extension that embeds state updates. Compared to SCSYNT, SmartSpec provides two advantages: (1) Conciseness. SCSYNT requires explicit update instructions for each state transition, while SmartSpec derives them automatically from functional specifications; (2) Interpretability. State-machine approaches invent internal state variables that obscure semantics, whereas SmartSpec uses only designer-specified states.

Sketch. Similar to the inductive phase of SmartSpec, Sketch [30] also uses a partial program as input, and synthesizes program that satisfy program correctness specification. However, we find two qualitative difference between SmartSpec and Sketch: (1) Sketch specializes in generating correct constants within arithmetic expressions, while SmartSpec synthesizes logical formulas in first-order relational logic. (2) Sketch typically requires a carefully constructed program sketch closely resembling the final program structure, while SmartSpec has the capability to generate the entire program structure through its deductive synthesis phase. While learning-based approaches [25] have shown promise in automatically synthesizing program sketches, they often demand substantial computational resources for training and may be limited in accuracy.

Summary. SmartSpec is both effective and efficient. It synthesizes correct smart contracts within seconds to minutes, achieves gas

efficiency comparable to hand-written implementations, and outperforms existing synthesis tools in automation and interpretability.

7 Related work

Declarative smart contract. DeCon [9] introduces a declarative approach to smart contract implementation, modeling contracts as database manipulation programs in Datalog and compiling them deterministically to Solidity. Inspired by DeCon, we extend this approach to the deductive sketch generation phase in our synthesis framework. SmartSpec strengthens correctness guarantees by verifying user-specified safety properties over infinite transaction sequences, a non-trivial challenge in contract verification [26] [8], whereas DeCon focuses on ensuring semantic equivalence between Datalog and Solidity. Additionally, SmartSpec integrates search and verification into the synthesis process, making it more complex than DeCon’s deterministic compilation, yet enabling the generation of verified contract implementations.

Smart contract verification. An efficient verifier plays a pivotal role in the inductive synthesis phase of SmartSpec. Verx [26] verifies safety properties specified in temporal logic. DCV [8] focuses on verifying safety properties for smart contracts written in declarative languages to facilitate efficient verification processes. SmartSpec adopts a similar specification interface as Verx and leverages the verification backend of DCV to achieve efficient verification.

Additionally, SmartSpec generates counterexamples when a candidate smart contract fails to satisfy specified safety properties. EthBMC [18] and ESBMC [31] specialized in finding vulnerabilities in smart contracts. These tools are designed to identify specific vulnerabilities, while the verifier in SmartSpec finds counterexamples for general user-specified safety properties.

Program synthesis. Program synthesis techniques [20] have been extensively studied. The inductive synthesis phase of SmartSpec is grounded in constraint-based techniques [22], where both correctness specifications and syntactic restrictions on the program are encoded in a single formula. This encoding enables a satisfying solution to correspond to a correct program. Constraint-based techniques have shown success in various domains including program sketching [30], Datalog program synthesis [5], Database schema refactoring [34], and network configuration synthesis [16]. The key innovation of SmartSpec is the decomposition of the smart contract synthesis problem, which exposes a sub-problem that is suitable for encoding for constraint solving.

8 Conclusion

We present SmartSpec, a multi-modal synthesizer for smart contracts. By integrating formal specifications with concrete examples, SmartSpec strikes a balance between specification accuracy and intuitiveness. And it is efficient, synthesizing **XX** benchmark contracts in seconds to minutes. The generated smart contracts exhibit comparable gas consumption to hand-written smart contracts, further affirming their efficiency and practical viability.

This study opens promising avenues for future research, including designing specification and synthesis techniques that account for contract interactions, and repairing vulnerabilities in existing smart contracts. Addressing these challenges will push the frontier of smart contract programming automation.

References

- [1] solidity. <https://soliditylang.org/>, 2024.
- [2] Truffle suite. <https://archive.trufflesuite.com/>, 2024.
- [3] SmartSpec artifacts. <https://anonymous.4open.science/r/submission1425>, 2025.
- [4] solidity. <https://vyperlang.org/>, 2025.
- [5] Aws Albarghouthi, Paraschos Koutris, Mayur Naik, and Calvin Smith. Constraint-based synthesis of datalog programs. In *Principles and Practice of Constraint Programming: 23rd International Conference*. Springer, 2017.
- [6] Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo MK Martin, Mukund Raghothaman, Sanjit A Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. *Syntax-guided synthesis*. IEEE, 2013.
- [7] Stefanos Chaliasos, Marcos Antonios Charalambous, Liyi Zhou, Rafaila Galanopoulou, Arthur Gervais, Dimitris Mitropoulos, and Benjamin Livshits. Smart contract and defi security tools: Do they meet the needs of practitioners? In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–13, 2024.
- [8] Haoxian Chen, Lan Lu, Brendan Massey, Yuepeng Wang, and Boon Thau Loo. Verifying declarative smart contracts. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, page 12, Lisbon, Portugal, April 14–20 2024. ACM.
- [9] Haoxian Chen, Gerald Whitters, Mohammad Javad Amiri, Yuepeng Wang, and Boon Thau Loo. Declarative smart contracts. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 281–293, 2022.
- [10] Xiaohong Chen, Daejun Park, and Grigore Roşu. A language-independent approach to smart contract verification. In *Leveraging Applications of Formal Methods, Verification and Validation. Industrial Practice: 8th International Symposium, ISoLA 2018, Limassol, Cyprus, November 5–9, 2018, Proceedings, Part IV 8*, pages 405–413. Springer, 2018.
- [11] Zhiyang Chen, Ye Liu, Sidi Mohamed Beillahi, Yi Li, and Fan Long. Demystifying invariant effectiveness for securing smart contracts. *Proceedings of the ACM on Software Engineering*, 1(FSE):1772–1795, 2024.
- [12] Coinpaper. Weekly losses from web3 exploits exceeded \$138 million, 2024.
- [13] Leonardo De Moura and Nikolaj Björner. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.
- [14] DeFiHackLabs. Gpu. https://github.com/SunWeb3Sec/DeFiHackLabs/blob/main/src/test/2024-05/GPU_exp.sol, 2024.
- [15] DeFiHackLabs. Jokinthebox. https://github.com/SunWeb3Sec/DeFiHackLabs/blob/main/src/test/2024-06/JokInTheBox_exp.sol, 2024.
- [16] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. Network-wide configuration synthesis. In *Computer Aided Verification: 29th International Conference, CAV 2017, Heidelberg, Germany, July 24–28, 2017, Proceedings, Part II 30*, pages 261–281. Springer, 2017.
- [17] Bernd Finkbeiner, Jana Hofmann, Florian Kohn, and Noemi Passing. Reactive synthesis of smart contract control flows. In *International Symposium on Automated Technology for Verification and Analysis*, pages 248–269. Springer, 2023.
- [18] Joel Frank, Cornelius Aschermann, and Thorsten Holz. {ETHBMC}: A bounded model checker for smart contracts. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2757–2774, 2020.
- [19] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.
- [20] Sumit Gulwani, Oleksandr Polozov, Rishabh Singh, et al. Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2):1–119, 2017.
- [21] Infosecurity Magazine. Cyber attacks drain web3.0, expose ethereum smart contracts, 2024.
- [22] Susmit Jha, Sumit Gulwani, Sanjit A Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 1*, pages 215–224, 2010.
- [23] Moez Krichen, Mariam Lahami, and Qasem Abu Al-Haija. Formal methods for the verification of smart contracts: A review. In *2022 15th International Conference on Security of Information and Networks (SIN)*, pages 01–08. IEEE, 2022.
- [24] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 254–269. ACM, 2016.
- [25] Maxwell Nye, Luke Hewitt, Joshua Tenenbaum, and Armando Solar-Lezama. Learning to infer program sketches. In *International Conference on Machine Learning*, pages 4861–4870. PMLR, 2019.
- [26] Anton Permenev, Dimitar Dimitrov, Petar Tsankov, Dana Drachler-Cohen, and Martin Vechev. Verx: Safety verification of smart contracts. In *2020 IEEE symposium on security and privacy (SP)*, pages 1661–1677. IEEE, 2020.
- [27] Oleksandr Polozov and Sumit Gulwani. Flashmeta: a framework for inductive program synthesis. In *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 107–126. ACM, 2015.
- [28] Mukund Raghothaman, Jonathan Mendelson, David Zhao, Mayur Naik, and Bernhard Scholz. Provenance-guided synthesis of datalog programs. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–27, 2019.
- [29] Xujie Si, Woosuk Lee, Richard Zhang, Aws Albarghouthi, Paraschos Koutris, and Mayur Naik. Syntax-guided synthesis of datalog programs. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 515–527, 2018.
- [30] Armando Solar-Lezama. The sketching approach to program synthesis. In *Asian symposium on programming languages and systems*, pages 4–13. Springer, 2009.
- [31] Kunjian Song, Nedas Matulevicius, Eddie B de Lima Filho, and Lucas C Cordeiro. Esmc-solidity: An smt-based model checker for solidity smart contracts. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 65–69, 2022.
- [32] Petar Tsankov, Andrei Marian Dan, Dana Drachler-Cohen, Arthur Gervais, Florian Bünzli, and Martin T. Vechev. Securify: Practical security analysis of smart contracts. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 67–82. ACM, 2018.
- [33] Chenglong Wang, Alvin Cheung, and Rastislav Bodik. Synthesizing highly expressive sql queries from input-output examples. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 452–466, 2017.
- [34] Yuepeng Wang, James Dong, Rushi Shah, and Isil Dillig. Synthesizing database programs for schema refactoring. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2019.
- [35] Yuepeng Wang, Shuvendu K. Lahiri, Shuo Chen, Rong Pan, Isil Dillig, Cody Born, Imad Naseer, and Kostas Ferles. Formal verification of workflow policies for smart contracts in azure blockchain. In *International Conference on Verified Software: Theories, Tools, and Experiments (VSTTE)*, volume 12031, pages 87–106. Springer, 2019.
- [36] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- [37] Jingyi Emma Zhong, Kevin Cheang, Shaz Qadeer, Wolfgang Grieskamp, Sam Blackshear, Junkil Park, Yoni Zohar, Clark Barrett, and David L Dill. The move prover. In *Computer Aided Verification: 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21–24, 2020, Proceedings, Part I 32*, pages 137–150. Springer, 2020.

A Verification

Given a candidate smart contract, SmartSpec validates it by first applying bounded model checking to discover counterexamples against the specified safety properties. If a counterexample is found, it is returned to the inductive synthesizer as a new negative example. Otherwise, SmartSpec proceeds to prove correctness against given specifications via mathematical induction on transaction steps.

Transforming smart contracts to state transition systems. SmartSpec adopts the techniques in DCV [8] to transform a smart contract into a state transition system. Here, contract states are modeled by the system states, and each transaction that updates contract states are modeled as a state transition, subject to the transition validation conditions and the state update instructions associated with each type of transaction.

Temporal logic to contract invariant. For each sub-term $\Diamond\phi$ (once ϕ) in a temporal logic formula, SmartSpec adds a variable to track the truth value of the term, *i.e.*, once the inner term ϕ is evaluated to true, it remains true in all future transitions. Similarly, each $\bullet\phi$ (prev ϕ) sub-term also has a dedicated variable to track the value of ϕ in the states prior to a transition. Through these transformations, the problem of temporal property verification is effectively reduced to a safety invariant verification problem.

Bounded model checking. Utilizing the state transition system and safety invariants derived from temporal logic formula, SmartSpec employs bounded model checking to explore a sequence of up to N transactions. This exploration aims to identify scenarios wherein the state transition system enters an unsafe state, thereby violating the specified invariants.

Default parameters of Ethereum transactions, such as message sender, message value, and block sequence as a timestamp, are automatically incorporated into each transaction in the counterexample search. An additional constraint on timestamp is added to maintain the validity of the generated counterexample: $\forall i \in [1, |T| - 1]. T[i].timestamp < T[i + 1].timestamp$, *i.e.*, timestamp increases monotonically in each transaction sequence.

Induction proof. SmartSpec adopts the mathematical induction approach from DCV [8] to prove safety invariants by modeling the smart contract as a state transition system, where transactions trigger state changes. The goal is to ensure that, starting from the initial states, no unsafe states are reachable. The key challenge lies in strengthening invariants to be inductive across transactions. DCV leverages relational abstraction for efficient invariant search. Once proven, the contract is returned with concrete Solidity syntax.

B Proof